Page (shown partially in listing 24.13) to use the `IAuthorizationService` to verify whether the action is allowed to continue execution.

Listing 24.14 Using `IAuthorizationService` for resource-based authorization

```
[Authorize]                                                          ❶
public class EditModel : PageModel
{
    [BindProperty]
    public Recipe Recipe { get; set; }

    private readonly RecipeService _service;
    private readonly IAuthorizationService _authService;             ❷

    public EditModel(
        RecipeService service,
        IAuthorizationService authService)                           ❷
    {
        _service = service;
        _authService = authService;                                  ❷
    }

    public async Task<IActionResult> OnGet(int id)
    {
        Recipe = _service.GetRecipe(id);                             ❸
        AuthorizationResult authResult = await _authService          ❹
            .AuthorizeAsync(User, Recipe, "CanManageRecipe");        ❹
        if (!authResult.Succeeded)                                   ❺
        {                                                            ❺
            return new ForbidResult();                               ❺
        }                                                            ❺

        return Page();                                               ❻
    }
}
```

❶ Only authenticated users should be allowed to edit recipes.

❷ IAuthorizationService is injected into the class constructor using DI.

❸ Loads the Recipe from the database

❹ Calls IAuthorizationService, providing ClaimsPrinicipal, resource, and the policy name